uses angle brackets:

```
let elements: HTMLElement[] = document.querySelectorAll(this.targetSelector)
let inputElements: HTMLInputElement[] = <HTMLInputElement[]>elements
```

An important fact about these type assertions is that they are compile time only—they are only there to give the compiler more information about the properties available on that data. If the data is incorrect at run time, you'll get a run-time error at the time you try to use a nonexistent property, not at the time of this type assertion. There is a way to do run-time type checks in TypeScript, which you'll see how to do in Chapter 13, *Validating Code with Advanced TypeScript*.

There is one other place in which you might have more information than the compiler. If you have functions that return functions, TypeScript may not be able to infer the type of the `this` parameter in the internal function. To combat that, any TypeScript function definition can define `this` as the first parameter in an argument list. Defining `this` explicitly allows you to give `this` a type, but does not affect the parameters that would be used in calling the function.